Testing some aspects of Object-Oriented Ontology using an Open Source Software Tool.

Robin Baines, July 2026

1 Abstract

Object-Oriented Ontology (OOO) is a recent philosophical movement that rejects human-centric thinking. Developed by thinkers like Graham Harman, Ian Bogost, and Timothy Morton, OOO emerged as a branch of Speculative Realism[1].
WOOOF is an open source program which converts a scenario into an object model. WOOOF has been used to test several OOO ideas using three scenarios which describe interactions between objects without an observer. A uniform approach was developed, comparable with application of a pattern in Object Oriented Programming. A brief description of Brownian Motion in this context raises some questions as to how univeral this approach may be.
Compound objects come into existence and disintegrate during scenarios. Avoiding internal hierarchy in these real objects avoids complications and supports the OOO idea of a flat ontology. However OOO seems to suggest that a real object could include another object and retain its identity. When considering, for example, coalescence of water drops this implies an arbitrary boundary on the relative size of drops. One approach is to avoid defining the exact nature of real object composition and to use relevant, or all, combinations of the sensual objects in the compound object.
In the scenarios, time is divided it into periods of gradual change punctuated by events when changes occur instantaneously. The events are used to implement the OOO idea of vicarious causation.
A second type of scenario introduces two observers, a human and a dog both of which perceive and influence each other. Parallel sub-scenarios are used to represent a delayed response of the human to an action taken by the dog and vice versa. A simple object model is introduced to organise pre-existing information in the observers.
The four scenarios support aspects of OOO but also include instantaneous events and introduce several objects types in observers.

1Speculative Realism rejects the idea that reality is restricted to or defined by human thought.

- **Anti-Correlationism:** "Correlationism," is the belief held since Immanuel Kant that humans can only access the correlation between thinking and being, never reality-in-itself.
- **Realism Without Humans:** We can think about a universe where humans do not exist, such as a time before life evolved.
- **De-centering Humans:** humans are not at the center of philosophy. The non-human world can be studied on its own terms.

Table of Contents

1 Abstract....1
2 Introduction....3
3 Using WOOOF to express OOO ideas....4
3.1 The INIT object....5
3.2 Composition....5
3.3 Inheritance....6
3.4 Properties....6
3.5 Time and Events....6
3.6 Object Lifecycle....8
3.7 Independent Scenarios....8
3.8 Level of Detail....9
3.9 Compound Objects....10
3.10 Bringing it together: drops scenario as an Example....12
3.11 Brownian Motion....13
3.12 Instantiation....13
4 A Scenario with Observers....13
4.1 The Dogs Scenario....14
4.2 Object hierarchy and Inheritance inside an Observer....15
4.3 Property Visibility....16
5 Do the scenarios support OOO?....17
5.1 Level of Detail and Withdrawal....17
5.2 Timelines and Events. Two types of object?....17
5.3 A Generic Approach to Inanimate Interactions....17
5.4 Compound Objects....17
5.5 Properties and OOO....17
6 If OOO does describe reality or, at the very least, is a useful analysis tool; what questions arise from its success?....18
6.1 Why do Objects Work Outside and Inside an Observer?....18
6.2 An 'interface' Object which Observers use when Interpreting Reality....19
7 Conclusion....19
8 Appendix: Brownian Motion....19

Table of Figures

Figure 1: drops.txt....3
Figure 2: golf.txt....3
Figure 3: PoolAndStone.txt....4
Figure 4: The Real Object called RO_bigdrop is included in the SO called bigdrop....5
Figure 5: The club_ball compound object....5
Figure 6: Presentation of Inheritance....6
Figure 7: My_Nida is a dog. dog is an animal....6
Figure 8: The drops scenario showing the INIT object, the collision timeline and the COLLISION Event....8
Figure 9: the club_ball object is Greyed out, the separate club and ball objects survived the scenario....9
Figure 10: PoolAndStone scenario showing Independent sub-scenarios: Air stops moving in pl1 and Stone falling in water in pl2....10
Figure 11: The Stone touches the Water and the stone_air_water Compound object is created....12
Figure 12: WOOOF output of drops.txt....14
Figure 13: SEE_SOMETHING black moving....15
Figure 14: My_Nida contains Today_Nida and Today_Nida is a type of My_Nida....16

2 Introduction.

Object-Oriented Ontology (OOO) is a recent philosophical movement that rejects human-centric thinking. OOO asserts that all objects exist independently of human perception and share an equal ontological status. An object is fundamentally "withdrawn" and never fully knowable or reducible. Developed by thinkers like Graham Harman, Ian Bogost, and Timothy Morton, OOO emerged as a branch of Speculative Realism.
See [1] Object-oriented Ontology: A New Theory of Everything by Graham Harman.
And [2] and [3] for details of OOO.
See [4] for an overview of Speculative Realism and how Object-oriented Ontology relates to other Speculative Realism ideas.

Several Object Oriented Ontology (OOO) ideas are:

- Real Objects (ROs) with Real Qualities (RQs) and Sensual Objects (SOs) with Sensual Qualities (SQs) exist.[2]
- Objects form a flat ontology implying, among other things, that interaction between inanimate objects without an observer is valid.[3]
- Vicarious causation; object interaction is facilitated by a third object.[4]
- Undermining; a real object cannot be reduced downward to only its parts.[5]
- Overmining; a real object has a surplus beneath the sum total of effects.[6]

WOOOF is an open source program which converts a scenario, written in the WOOOF language, into an object model. The program and the scenarios mentioned in this article are available from https://github.com/RobinBaines/OOO3/releases/tag/v1.0.4. The handbook is in the Readme.md file on the site.
The WOOOF language uses Object Oriented Programming (OOP) ideas Inheritance, Properties and Composition. WOOOF can be used to experiment with different approaches to modelling objects and object interaction. I have used the WOOOF program to test several OOO ideas in three scenarios, modelling interactions between inanimate objects without an observer:

1. drops.txt Two drops of water coalesce. The resulting drop splits in a gust of wind.

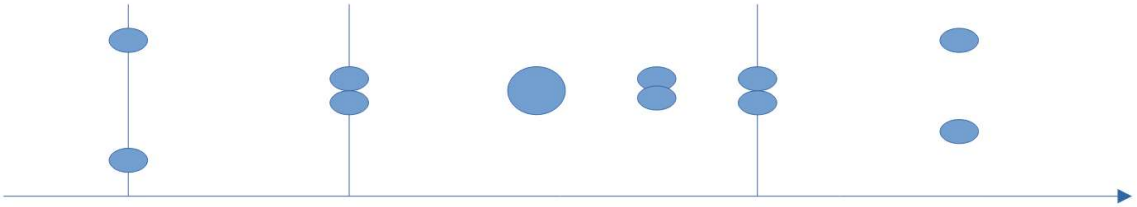


Figure 1: drops.txt

2 An object is anything that can be thought about, perceived, or acted upon by a subject. In OOO a real object is fundamentally "withdrawn" and never fully knowable or reducible. Sensual objects exist only when they are perceived, imagined, or encountered by another object.

3 A flat ontology of real objects means that objects have no vertical hierarchy in the real world, which feels logical; after all accepting objects as being real is difficult enough but trying to divide all these object into different categories by adding a property or using inheritance and claiming that all objects fall neatly into a single category feels like a step too far. A flat ontology avoids discussions about categories. A challenge is that very different objects, for example, a golfclub and Sherlock Holmes, are equivalent.

4 A consequence of withdrawal of real objects in OOO is that they cannot touch directly and that raises the question of how real objects interact. Vicarious causation uses an intermediary object to direct the interaction. When discussing this with friends a theatre came to mind.

5 Reducing a real object downward to only its parts misses properties which emerge from the interaction of the parts. This is also related to (weak and strong) emergence. Weak emergence means that an object has properties which exist because of the interaction between its parts and which can, in principle, be deduced from the parts. In general the deduction is based on Physics. An example is water having parts hydrogen and oxygen. Many properties can be deduced but are, at the same time, not immediately obvious when looking at each part separately. Strong emergence states that a whole system has properties that its individual parts do not share, and that these properties cannot be deduced or predicted from the parts, even in principle. Strong emergence is more problematic and some may claim that it does not exist.

6 Avoiding under- and over- mining requires that an object is more than its parts and less than its effects.

2. golf.txt A golf club hits a golf ball and the club and ball form a new object. After a period of intense energy transfer the club and ball separate. [7]

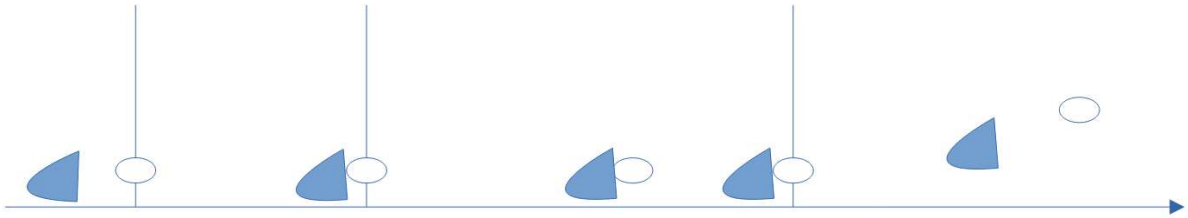


Figure 2: golf.txt

3. PoolAndStone.txt A stone falls towards the surface of a pool of water.
The stone enters the water and comes to rest at the bottom of the pool.
The air which was disturbed by the passage of the stone comes to rest.
The water which was disturbed by the passage of the stone comes to rest.

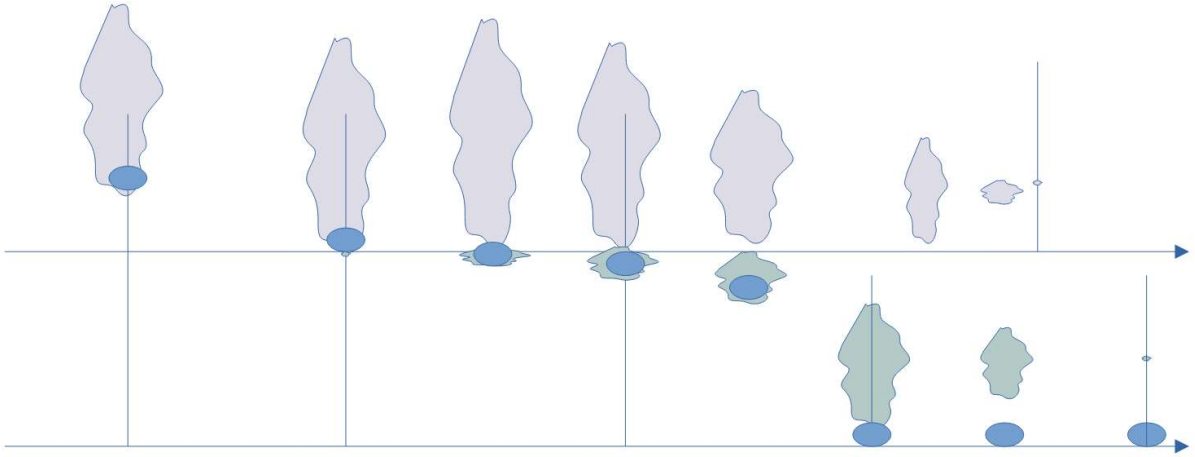


Figure 3: PoolAndStone.txt

A forth scenario, dogs.txt, introduces two observers, a human called 'Me' and a dog called Nida.
A dog is sitting then running and then barking at an area where dogs can run freely.
The human recognises the running dog and later hears it barking.
The human commands the dog to sit.
The dog recognises the command, stops barking, stops running and sits.

The following scenarios are part of the WOOOF release on github.

- ACW.txt describes the American Civil War and is an early script which was inspired by Graham Harman's analysis and ideas on object development using symbiosis.
- PoolAndStoneObject.txt is a version the PoolAndStone.txt script using an alternative approach to compound objects.

The first chapter describes how WOOOF has been used to express OOO ideas in the three scenarios without an observer. The second chapter shows the effect of adding an observer. The third chapter addresses whether the scenarios support OOO. The last chapter considers the implications of OOO being an accurate description of reality.

3 Using WOOOF to express OOO ideas.

The WOOOF language is used to define a scenario and the WOOOF program is used to display it. The scenarios can be expressed using the language in several ways and finding scenarios which had a common approach and which apply OOO ideas consistently involved several revisions. The following summarises the findings.

7 See https://www.youtube.com/shorts/md_CSMNegXI for an example of super slow motion of a golfclub hitting a ball.

3.1 The INIT object.

The INIT object is the first object in a scenario and is used to introduce and initialise other objects. This convention avoids having to describe how each object has come into existence. For example it would be tedious to describe how the two drops in drops.txt came into existence via a series of events involving nucleation, condensation and collisions. The INIT object is useful for practical reasons related to scripting. It has similarities with the definition of initial conditions in engineering models.

3.2 Composition

In WOOOF composition uses the 'includes' and 'include_right' commands. For example ‘The collision includes drop1’ in the drops scenario and ‘Me includes My_Nida’ in the dogs scenario. In OOP composition can also be expressed as an Object ‘has another’ object; for example, a car has a steering wheel. The include command has been used to implement the OOO idea that an object can have parts which are also objects and has been used to express the relationship between an SO and an RO. Figure 4 shows how WOOOF presents “bigdrop includes RO_bigdrop” in the drops scenario.

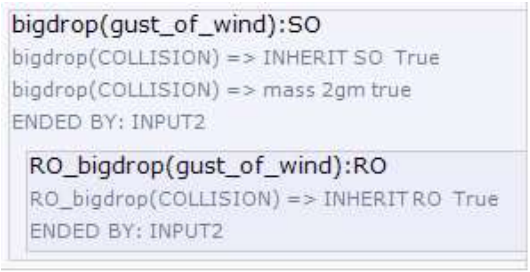


Figure 4: The Real Object called RO_bigdrop is included in the SO called bigdrop.

The SO bigdrop acts as a ‘wrapper’ object which encapsulates the RO called RO_bigdrop and manages external access. This is a reasonable match with the OOO concept of the withdrawal of real objects.
In OOO an object which is composed of two or more other objects is called a Compound object, see the following figure of the club_ball Compound object which occurs in the golf.txt scenario.
Figure 5 shows how WOOOF presents the following in the golf.txt scenario.

club includes RO_club.
…
ball includes RO_ball.
…
club_ball Includes RO_club_ball
RO_club_ball includes club
RO_club_ball includes ball

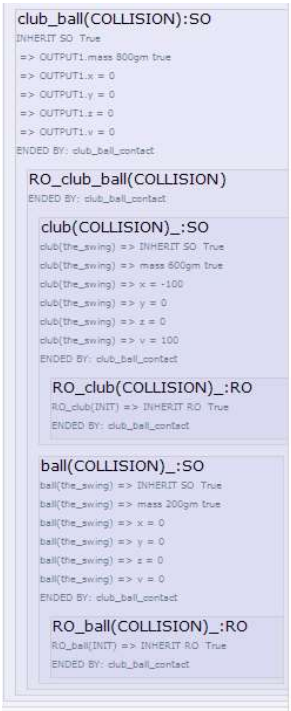


Figure 5: The club_ball compound object.

3.3 Inheritance.

Inheritance is an OOP term: Child classes may inherit (or be derived) from a parent class. The children are usually ‘a type of’ the parent; or the child ‘ís a’ parent. For example: A sparrow is a type of bird, or a sparrow is a bird.
WOOOF supports inheritance using the 'type' or 'type of' command and shows it as Child and Parent separated by a colon, see the following figure.



Figure 6: Presentation of Inheritance

The figure shows how “COLLISION is a type of SO_EVENT” in the drops and golf scenarios.

3.4 Properties

The scenarios show how objects could relate to each other and to keep them short, properties have been kept to a minimum. Scripts of inanimate interactions include coordinate and velocity properties chosen to ensure that the drops collide. In retrospect this looks like an unnecessary part of the scenario.

3.5 Time and Events.

Time increases from top to bottom in the text file in which a scenario is defined and from left to right in the WOOOF display.
Time has two implementations:

- A 'Timeline' is a Sensual Object with a finite duration. It is derived from an object called RO_timeline, which is a single RO representing the time continuum.
- A discrete ‘Event’ is a Real Object with no duration and is unique. ("en un momento dado"[8]; Johan Cruijff). An Event is instantaneous and therefore has an input with no more than two objects and an output, also, with no more than two objects. Potentially there could be a none countable, infinite number of Events. In practice an Event is instantiated when an object ends or when an object becomes part of a new object or if a property of an object changes. In scenarios Events names are in capital letters. [9]

An Event may have an input and/or an output object called INPUTx and OUTPUTx, by convention; see the drops scenario in the following figure. As described above, an RO is contained in an SO; the Real Object called RO_COLLISION is contained within the SO called COLLISION.

8 Although the expression already existed in Spanish it achieved iconic status (in Spain and the Netherlands) when Cruijff used it during his period as trainer of FC Barcelona from 1988.

9 In every day use the word ‘event’ is most often used to describe a visit to the theatre or a football match. In WOOOF the meaning is closer to the idea of an event in a C# program, for example, a mouse click or a file has just been read. An alternative name is Operator: In quantum mechanics, operators represent mathematical instructions like measuring position or momentum and they act instantaneously.



Figure 7: The drops scenario showing the INIT object, the collision timeline and the COLLISION Event.

Events occur at the beginning and end of a Timeline and it is the time stamps of these Events which mark the start and the end of the enclosed Timeline.
This approach to time and the introduction of the Event avoids simultaneous existence of objects which enter an Event and the modified objects leaving the Event. For example, in the drops scenario, two drops enter THE_COLLISION Event and one coalesced drop, called bigdrop, leaves it. In other words the Event avoids the two seperate drops and the coalesced drop existing at the same time.
In the scenarios an Event is the vicarious RO which facilitates interaction between objects. In some other scenarios very detailed Events may be justified. For example Brownian motion and a drops scenario with atomisation, instead of coalescence, would be scripted using many Events separated by short Timelines, the assumption being that no two Events occur in the same instant, see the chapter 3.11 Brownian Motion.

3.6 Object Lifecycle.

If an Object, for example My_Nida in dogs.txt, has been included in a TimeLine and is then included in the next Event, the My_Nida SO is reproduced by the WOOOF program and the version in the Timeline SO is greyed out. As the object proceeds through the Timelines and Events its properties change but the copies in preceding Timelines and Events do not. We can think of the greyed out objects as 'snapshots' made at the moment the containing Timeline or Event ends. If an object does not end within the scenario it will not be greyed out in the last TimeLine; it has survived the scenario!
The following figure shows the Compound object club_ball greyed out in the SPLIT Event. After the split, the separated club and ball objects still exist and are therefore not greyed out.



Figure 8: the club_ball object is greyed out, the separate club and ball objects survived the scenario.

3.7 Independent Scenarios.

Several independent scenarios can occur within the main scenario. This idea is used in the PoolAndStone script: At a certain moment the stone is fully submerged and it continues to fall through the water. Independently, the air, which was disturbed earlier by the passage of the stone above the water, slowly comes to rest. The stone sinking in water and the air coming to rest are two independent scenarios and are displayed in two independent ‘placeholder’ objects. See pl2 and pl1 in the following figure.

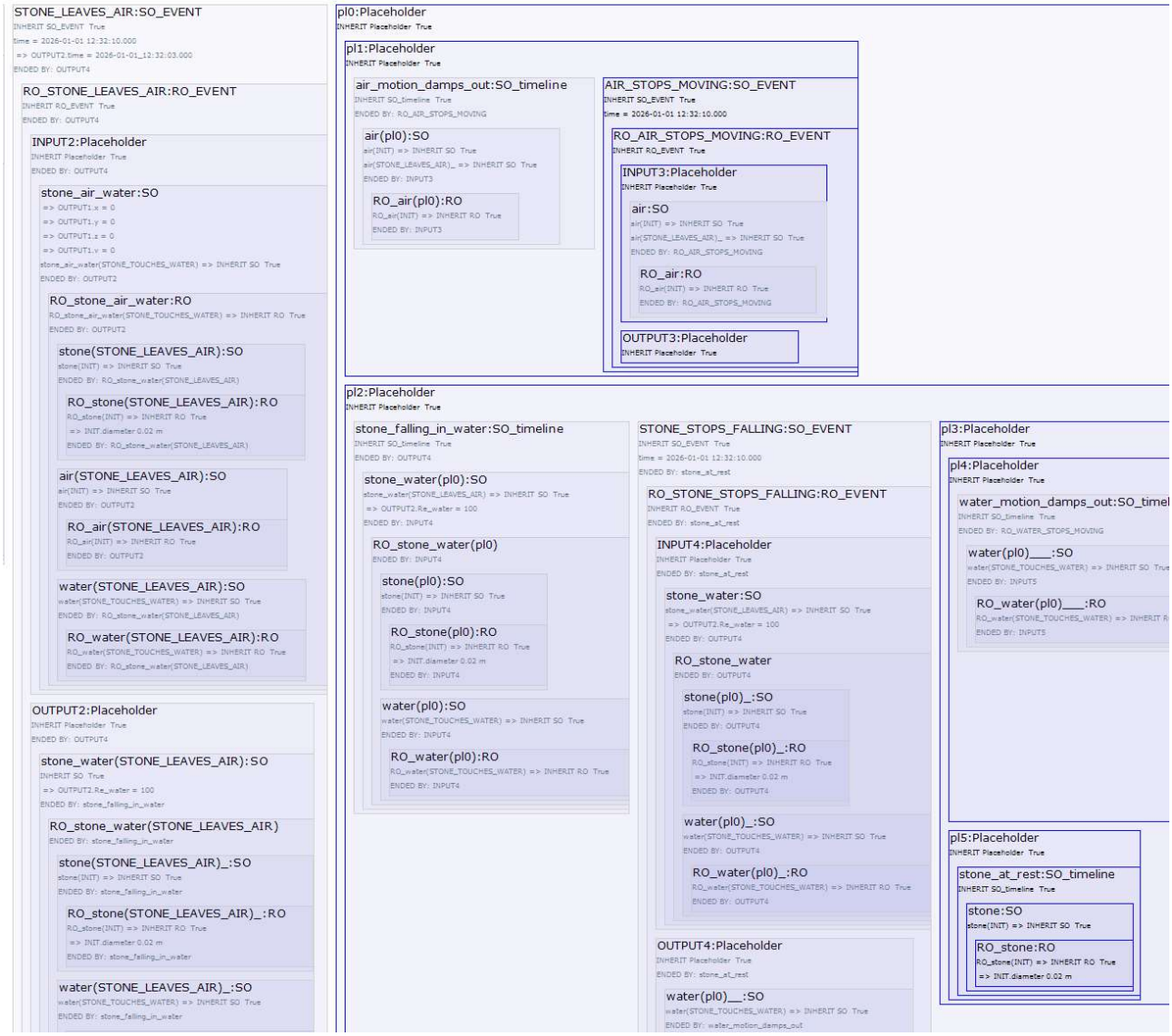


Figure 9: PoolAndStone scenario showing Independent sub-scenarios: Air stops moving in pl1 and Stone falling in water in pl2.

3.8 Level of Detail.

Timelines and Events are related to paradoxes associated with Zeno of Elea; if we are tempted to dive into ever more detail in a scenario then we could end up with so many events that no progress is made. The OOO idea of withdrawal is also a warning that this is not a good approach and what we may be attempting to achieve will remain just out of reach.

Here are some examples of how Events have been used in the scripts:

- In the drops script two drops of water collide and form a new drop in the Event called THE_COLLISION.
- In the golf script the club and ball make contact in an Event which is also called THE_COLLISION and form a new compound object called club_ball.
- In the PoolAndStone script the Event, called STONE_TOUCHES_WATER, is the moment the stone makes contact with the pool of water.

Defining the moment an Event takes place has some similarity with finding a maximum or minimum on a smooth curve without using differentiation. We know there is a maximum and can choose a position on a curve before the maximum and another after. Then by stepping from both directions the length of the curve on which the maximum must be gets shorter and shorter. In a similar way, in the drops script, we know there must be an instant when two drops coalesce to form a single drop, but have trouble finding it and defining the exact timestamp; which may have an infinite number of decimals!

What then should the level of detail be in a scenario? The division of time into Timelines and instantaneous Events, has the potential for introducing ever more detail. In a similar way other types of object can be split into ever more detail, potentially to atoms and further. When considering inanimate scripts it looks difficult to fix this granularity without slipping in an observer to define the intention of the analysis. An observer could, for example, conclude that there is no point in creating Events for the arrival of every photon on the surface of raindrops in the drops scenario. Events seem also to be associated with a change in the rate of energy transfer and the same conclusion to ignore photons, is reached by comparing the rate of energy transfer caused by a photon and the much greater rate caused by colliding water drops.
Changes in the structure of objects and step changes in the rate of energy transfer seem to be pointers to a consistent level of granularity and seem to imply that the OOO concept of withdrawal applies to time objects as well as physical objects. Withdrawal can be seen as an incentive to finding a practical point at which to stop digging deeper.

3.9 Compound Objects

In scenarios, Compound objects are formed in Events and because an Event is instantaneous a compound object is formed from two objects, which may also be Compound objects.
In the drops scenario the Compound drop formed when 2 smaller drops of similar size, coalesce is a new object called bigdrop. Initially drop1 and drop2 are recognisable parts of bigdrop, recognisable by the shape which resembles 2 fried eggs lying next to each other in a frying pan. However rather quickly surface tension and mixing cause the original drops to lose their identity and the new bigdrop dominates any subsequent interactions. We can conclude that immediately after first contact drop1 and drop2 exist but later they do not. It is reasonable to conclude that drop1 and drop2 meet their demise at exactly the same moment in an Event which is bound to occur but defining when exactly it occurs is impossible. We can conclude that Sensual Objects drop1, drop2 and new bigdrop and all combinations, exist from the moment the collision occurs but that only Sensual Object bigdrop survives.
OOO seems to suggest that an object could include another object and retain its identity if its essential nature was retained. We could let drop2 merge into drop1 or the other way around. However with 2 drops of similar size choosing one or the other look arbitrary. Does that mean that any coalescing drops should always form a new drop even when one drop is significantly larger than the other? If this were not the case then an arbitrary, relative size would be needed to establish when the larger drop is so much larger that it absorbs the smaller one.
In the golf scenario club and ball both start deforming and energy starts to be transferred at the moment the collision Event occurs. By analogy with the drops scenario the club and ball form a new Compound object in the short period of contact and energy transfer. Alternatives are to let the club include the ball or the ball the club. Choosing one or the other looks arbitrary and does not improve or simplify the scenario.
The initial collision Event in the drop and golf scenarios have many similarities. It is only after the Event has taken place that the properties of drops, club and ball manifest themselves resulting in coalescence in one case and intense energy transfer and distortion but with retention of object integrity, in the latter case. Similarily the difference between these scenarios and, for example, a scenario where a club hits a raindrop only becomes evident after the initial Event.
Properties exercise influence after an Event in inanimate interactions and we could identify the relevant property in these scenarios as, “the potential to atomise” or “character of a liquid”, or “ability to retain object integrity during intense energy transfer”.

In his book [2] Waves and Stones: On the Ultimate Nature of Reality, Graham Harman describes how a stone is thrown into an approaching wave. The Pool and Stone scenario is similar.
Here is the simplified scenario:

1. A stone falls through air towards a pool of water.
2. It touches the water and moves into it until it is fully immersed.
3. It falls to the bottom of the pool.
4. Water and air motion caused by the passage of the stone damp out.

The water and air objects are defined as the volumes in motion because of the passage of the stone. Here is the above PoolAndStone scenario including these objects:

1. Before the stone touches the water, the air and stone are a Compound object called stone_air and the water object does not exist.
2. When the stone touches the water, the air, water and stone form a new Compound object called stone_air_water.
3. The water object has no volume when it is created and grows rapidly as the stone moves into and through the water.

Before contact with the stone, the pool is a continuum. On contact with the stone the water object

becomes a discrete, increasing, volume as the stone enters and moves through the water.
4. When the stone is just fully immersed, the air object separates from the stone_air_water and becomes an independent object.
Objects water and stone continue as a new Compound called stone_water.
5. The air object begins to shrink as viscosity damps the disturbances caused by the passage of the stone. (Interaction between air and water at the water surface and interaction between the air object and the surrounding air are ignored).
6. Eventually the air is at rest and the air object has zero volume and ends.
7. When the stone comes to rest at the bottom of the pool, the stone_water Compound object ends and the water and stone objects separate.
The stone comes to rest and the water and stone compound object splits into separate water and stone objects because there is no longer interaction between them, or to put it another way, the stone no longer transfers energy to the water.
8. The water object begins to shrink as viscosity damps the motion caused by the passage of the stone.
9. Eventually the water is at rest and the water object ends: All the water in the pool is again part of the continuum.

All Compound objects are 'new' objects in the PoolAndStone scenario. For example, stone_air is a new object which includes stone and air and stone_air_water is an object which includes stone, air and water, see the following figure.

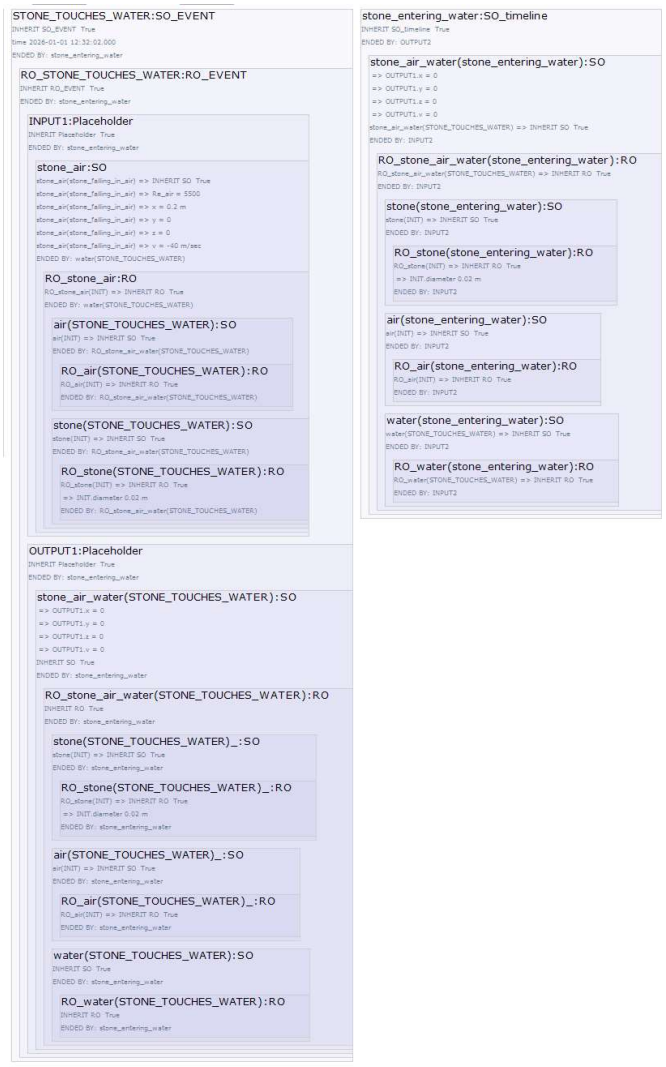


Figure 10: The Stone touches the Water and the stone_air_water Compound object is created.

The PoolAndStoneObject.txt script is an alternative PoolAndStone scenario and tests the idea of defining a dominant object and merging other objects into it. Instead of making Compound objects, stone_air, stone_air_water, stone_water and stone, the stone is given a 'dominant role'. The stone retains its identity as

a stone and air and water are included and excluded as the script progresses. This approach simplifies and feels natural because the stone includes entrained air and water when they are still in contact with the stone. However if the air – water interaction at the surface of the pool were included in the script complications would arise if these damped out after the stone came to rest at the bottom of the pool. In a shallow pool it is likely that energy transfer between water and air, as ripples spread across the pool, is significant after the stone has come to rest. Similar complications occur when trying to make object air or water play a 'leading role'. Another approach is to include stone_air as a Compound object in the object stone_air_water. This would lead to problems if properties depend only on air and water or stone and water.
These considerations imply that Compound objects are created by combining other objects to form a new object without internal hierarchy. This approach does not involve decisions on which object is dominant or problems when one object leaves a compound object which may itself be part of another compound object. For example 'air' has to leave stone_air_water which is a compound of stone_air and water. However this decision comes with a fairly serious objection; a collision between a large drop and an extremely small one must result in a new object and likewise the result of a rain drop falling into a large pool of water must also be a new object.
As mentioned above, these problems can be avoided by not defining the exact nature of real object composition and to use relevant, or all, combinations of the sensual objects in the compound object[10].

3.10 Bringing it together: drops scenario as an Example

The above points are illustrated using the drops scenario, see Figure 1, which shows two drops colliding and then splitting. The following figure shows the WOOOF presentation of the collision.

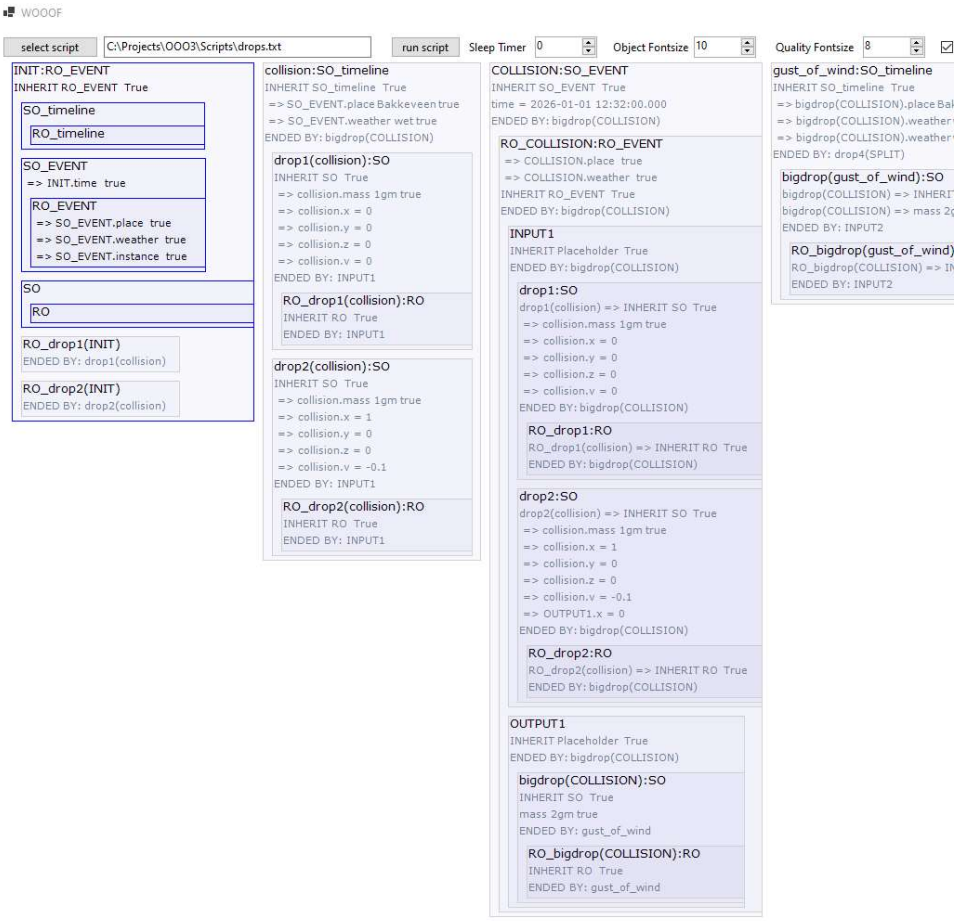


Figure 11: WOOOF output of drops.txt

The first block shows the INIT Event which introduces several object the most important being the two drops. The 2nd block shows the 'collision' Timeline which captures the movement of drop1 and drop2 on a collision

10 If 2 real objects in a compound object have sensual properties which depend on both objects then an SO combining both objects would be appropriate.

course. These drops include respectively RO_drop1 and RO_drop2. The 3rd block shows the COLLISION. RO_COLLISION is included in the SO COLLISION and the vicarious causation is shown included in the RO_COLLISION. The INPUT of the Event is drop1 and drop2 and the OUTPUT is bigdrop, an SO which includes the newly created bigdrop and the real object, RO_bigdrop.

3.11 Brownian Motion

Brownian motion is another example of inanimate interaction. In 1827, while examining grains of pollen suspended in water under a microscope, Robert Brown observed minute particles ejected from the pollen grains, executing a continuous jittery motion. He then observed the same motion in particles of inorganic matter, showing that the effect was not caused by something living.
In 1905, Albert Einstein published a paper explaining that Brownian motion was caused by invisible water molecules constantly colliding with the larger suspended particles, providing definitive physical evidence that atoms and molecules exist.
Some sizes, times and collision numbers are in the appendix. In that example a silica particle is suspended in water at room temparature (293 K). The Total Collision Rate of water molecules on the surface of the particle is extremely high (10^{19}/second), but almost all the collisions cancel each other out. The Net Collision Imbalance is the instantaneous difference in the number of water molecules striking opposing sides of the particle. Usually this is around zero but statistical asymmetry results in the occasional imbalance and a fluctuating net force that drives Brownian motion. Compare with tossing a coin; a long series of heads is unlikely but if enough tosses are made it will occur.
Using averages, about 3160 million molecules hit the particle between any 2 Brownian imbalances. Simulation would require double so many Events (one for a collision and one as the molecule leaves the surface of the particle) and compound object mutations.
This shows that simulating Brownian motion using Events and Timelines and WOOOF is not practical. The problem posed by Brownian motion occurs because of the extreme difference between the mass of the silica particle and the molecules of water while both are relevant.

3.12 Instantiation

Data processing in OOP often uses class instantiation. Consider a class describing an invoice (unique invoice number, date, sender, receiver, a list of products being billed, ...). When a new invoice is created an object is instantiated from the class using parameters such as invoice number, date, sender etc. The resulting invoices are unique but all have the structure and properties defined in the class.
A similar approach works in OOO. Imagine many pairs of drops colliding and coalescing in a thunder storm and a version of the drops script with only coalescence and no drop splitting. This drops script could be converted into a class with parameters drop sizes, collision time, coordinates, velocities, etc., and could be instantiated for each of the pairs of colliding drops. It is also possible to imagine the output of two of these scripts being used as the input for another, creating a chain of small drops coalescing to form rain.
The scenarios described here are all static classes meaning they cannot be instantiated or to put it another way they are both a class and a single object.

4 A Scenario with Observers.

In [3] Graham Harman explains how an observer would visualize a tree. “the pine tree and I are separate objects residing on the interior of a third: the intention as a whole.” The pine tree residing on the interior of the intention is an SO of the real pine tree. The ‘I’ is at that moment busy watching the tree and is an RO.” And: “Finally, the intention as a whole must be classed as a real object rather than a sensual one….., the intention itself is in fact underway, quite apart from whether it relates to anything outside.”

My interpretation of the first part of chapter 3 on the American Civil War in [1] is that

The American Civil War is a Real Object.
The Real Object called Graham Harman has an extensive and detailed Sensual Object of that War.
He has written a Real Object which is the first part of chapter 3 of [1] on that War.[11]
The Real Object ‘Me’ had an Sensual Object of the War before reading [1]. (Limited reading.)
I augmented my American Civil War Sensual Object while reading the first part of chapter 3 of [1].
I used that knowledge and internet to create the Real Object scenario called ACW.txt.[12]

11 While he was writing we can imagine that the RO of Mr Harman and his SO of his knowledge of the American Civil War residing on the interior of the Real Object representing his intention to write what he knows.
12 And while I was writing the scenario I had a Real Object representing my intention which contained the Real Object Me and my American Civil War SO.

Based on what could be going on in an observer, I have written a scenario called the dogs scenario with two observers, a human and a dog. Parallel sub-scenarios are used to represent a delayed response of the human to an action taken by the dog and vice versa: The actions of the dog occur in one sub-scenario and the delayed response of the human, in another sub-scenario. ‘Abstract’ parent objects like ‘animal’ and ‘dog’ and interface objects which process information play a role in the script.
Inanimate Events involve object collisions, object splitting or object property changes. Events in observers involve other types of interaction such as speech, responses to sight, smell, sound, feel and interactions motivated by memory or imagination.

4.1 The Dogs Scenario.

The dog sits then runs and later barks at the same time as running. The human observer only sees the dog when it has started running and then later hears it barking. The human observer is a separate 'placeholder' with Timelines and Events which mirror the dog Timelines and Events but with a delay to represent the human observer seeing these Events shortly after they have occurred. The human observer, the 'placeholder', is called 'Me' in the scenario and what follows. The delay between something happening and ‘Me’ seeing it, is modelled using the following Events and Timelines, see next figure:

Event: SEE_SOMETHING black moving.
Timeline: something_running
Event: SEE_RUNNING_DOG: Identify it as a running dog.
Timeline: dog_running
Event:SEE_NIDA_RUN: It is the dog called Nida. Change the property of 'Today_Nida' to running.

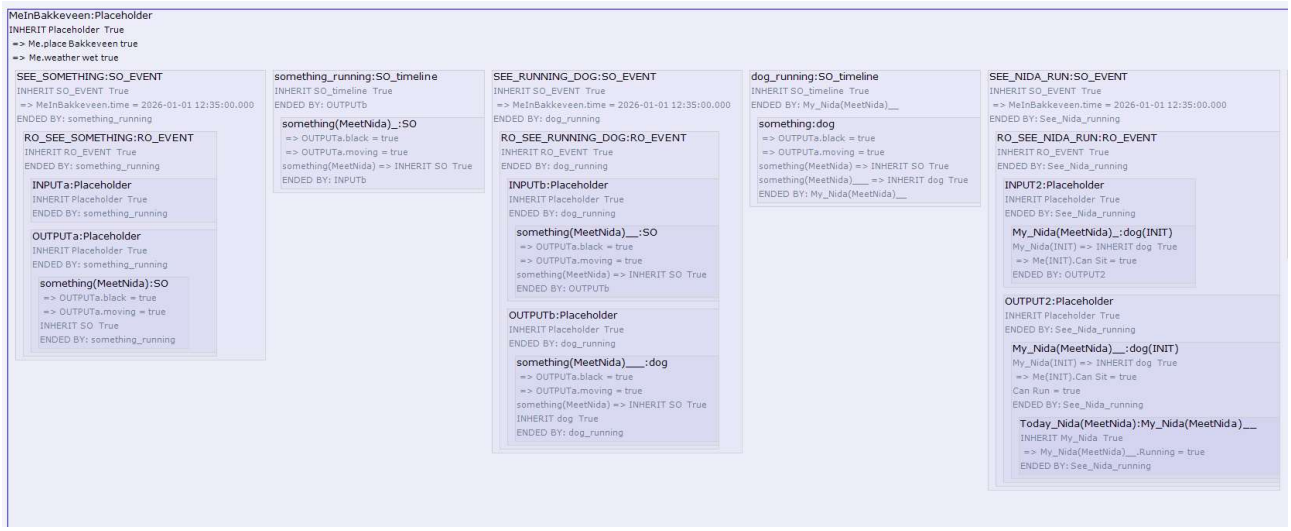


Figure 12: SEE_SOMETHING black moving.

Later the 'Me' object issues a command to sit. The dog interprets the command using a similar series of Events and eventually sits.



Figure 13: My_Nida contains Today_Nida and Today_Nida is a type of My_Nida.

The script uses a My_Nida object to capture everything 'Me' already knows about Nida. When 'Me' identifies the dog Nida he creates a new object called Today_Nida which tracks his experiences of Nida on the visit to Bakkeveen (a place in Friesland). Today_Nida is included in My_Nida as well as being a type (child) of My_Nida, see the following figure.
Inclusion of Today_Nida in My_Nida feels correct because the experience of My_Nida on that day are a part of My_Nida but are also separate because they relate to the events of that day. On completion of this part of the scenario My_Nida includes an encapsulated memory of the days events stored in Today_Nida.
If ‘Me’ sees Nida do something for the first time, for example jump 2 meters into the air, the Today_Nida would record this using an Event and would then update My_Nida, in another Event, with a new property: "can jump 2 meters into the air".

4.2 Object hierarchy and Inheritance inside an Observer.

Inheritance is used to organise observer information, for example, the human has an object ‘animal’ which is parent to the objects ‘human’ and ‘dog’ and ‘dog’ is parent to a dog called My_Nida. The object hierarchy as shown by WOOOF is shown in the following figure.



Figure 14: My_Nida is a dog. dog is an animal.

Storing properties in a parent object, such as animal, which apply to child objects dog and human, is an efficient way to store information and makes it available for rapid interpretation of new experiences.
I have discussed this with a friend who is an orthopedist. He thinks that some patients with an autistic condition are not always able to create these generic objects resulting in confusion if a new dog appears or that they are sometimes unable to use information about, for example, a red bus when they see a yellow one.
In OOP a parent object may be an abstract object implying that it cannot be implemented. For example if the parent is 'dog' then a child object may be an actual dog called My_Nida. The 'dog' would be abstract if we accept that there can be no actual 'dog' walking around and that any real dog is always a child of 'dog'.
In OOP one object including another object using composition implies a loose coupling between the objects. Inheritance implies a tight coupling and a tight coupling implies more changes to software when requirements change. WOOOF is useful for testing alternative approaches. In some earlier versions of the dogs scenario, information in an observer was represented using composition instead of inheritance. Inheritance results in a more natural implementation and can also be used to show that the observer, called 'Me', realises he is a type of 'human'.

4.3 Property Visibility.

In OOP languages such as C#, properties of a parent class may be Private or Public. Private properties are not accessible from the child class but Public properties are.
At this time there is no explicit Private/Public modifier in WOOOF but properties can be overridden. The parent 'classes' such as animal, dog and human have properties which apply to most of the derived objects, for example, a 'dog' represents what I may know about dogs and could have properties:

```
dog has the property    BackLeftLeg
dog has the property    FrontRightLeg
dog has the property    BackRightLeg
dog has the property    Barks [= true    //true is assumed if there is no true or false specified.]
```

If My_Nida does not bark the bark property of dog is overridden
    My_Nida has property Bark = false
If this override is not present then My_Nida inherits the Barks property from 'dog'; she barks!

5 Do the scenarios support OOO?

5.1 Level of Detail and Withdrawal.

The discussion of level of detail suggests a practical approach. When writing a scenario for collision of rain drops there seems little point in creating Events for the arrival of every photon on the surface of a drop. The same conclusion is reached by comparing the rate of energy transfer caused by a photon and the collision of rain drops. Practical limitations when writing scenarios or limitations introduced because of the intention of analysis do not influence the level of detail that reality might apply if real objects do exist. Brownian motion is difficult or perhaps impossible to simulate using WOOOF but this does mean that OOO ideas do not apply. These considerations support the OOO idea of withdrawal and application to all objects including the time objects introduced in these scenarios. In a certain sense we could turn the question around, realising that OOO encourages practical scenarios because of its acceptance of withdrawal.

5.2 Timelines and Events. Two types of object?

The Timelines have been implemented as SOs and as chunks of time derived (inherited) from the single Real continuous Timeline. Using SOs looks logical particularly when Timelines overlap when parallel independent sub-scenarios are part of a scenario.
Events are real objects which occur instantaneously. Instantaneous implies that they have an input with no more than two objects and an output with no more than two objects, avoiding complications.
However it does look as though this introduce two types of time objects in addition to the single object type proposed by OOO.

5.3 A Generic Approach to Inanimate Interactions.

Dividing time into Timeline and Event objects and the use of less obvious types of SO in the PoolAndStone script both contribute to a uniform approach to inanimate interactions. A uniform approach supports OOO and could be considered a requirement for OOO. Extending this uniform approach to Brownian Motion looks impractical using WOOOF but this does not mean that OOO ideas do not apply or that the uniform approach is not occurring.

5.4 Compound Objects.

OOO suggests that an object could include another object and retain its identity if its essential nature is retained. When considering, for example, coalescence of water drops this seems to imply that there is, what looks like an arbitrary boundary, on the relative size of drops: At some arbitrary point the resulting drop is not a new one but the large existing one with a small addition.
Brownian motion shows how many mutations to compound objects can occur in a fraction of a second.
This and withdrawal are perhaps telling us to avoid defining the exact composition of real objects and to use relevant, or all, combinations of the sensual objects in the compound object

5.5 Properties and OOO

Throughout the testing, finding reasonable and realistic properties was a problem particularily in the three scenarios without an observer. In [3] Graham Harman proposes three varieties of what he calls black noise[13] properties using a tree observed by a human as an example:

- Sensual tree has pivotal or essential qualities which identify it from moment to moment as the same tree.
- Accidental features which change from moment to moment (leaves moving in the wind, shadows falling).
- Properties related to other objects around it (other trees, mountains, mist).

Any golfer knows that trying to mark the position of a ball in the rough by coupling it to a particular tree in a group of several trees can be difficult. After moving 20 or 30 meters and then trying to identify that tree with certainty is difficult; what you thought were *essential* qualities of your chosen tree have become *accidental* ones.

13 As opposed to white noise which would imply something more random than these properties.

In [4 location 61%], causality is related to properties which are manifested under particular circumstances:

> "the causal powers possessed by things are entirely real. Salt is soluble; my drinking glass is fragile; this knife is sharp. Such qualities may only be manifested under particular circumstances; nonetheless, these "inherent causal powers" are real, intrinsic properties of the objects that bear them.…
> It is only in encounters with other objects that these powers are enacted.
> Causality thus exists objectively in the world; it is not just inferred (or imposed) by an observer."

In retrospect the use of scientific, engineering properties looks like an unnecessary part of the drops scenario and separating this type of property from the sensual object, feels artificial. Predicting coalescence or atomisation before a collision occurs also looks unnecessary because any instantaneous collision between two drops always results in a single drop. The Weber number of a single drop does look useful because it can predict whether a drop (formed by a collision) is unstable and is likely to split.[14]
When comparing the drops and the golf scripts it looks reasonable to conclude that a drop has the potential to atomise. While a golfclub or a ball could atomise the chance that this property is manifested is far less.

Another aspect of properties is illustrated in the dogs scenario. The human observer recognises a dog as My_Nida, the dog he already knows. A new version of Nida called Today_Nida is instantiated with My_Nida as parent. In OOP some of the properties of the parent may be made available to the child, see 4.3 Property Visibility. This idea looks logical in the scenario; My_Nida's colour and more abstractly the properties of my relationship with her, do not need to be rediscovered, they are made available when the existing object My_Nida is recalled when I recognise Nida.
The above list of property varieties could include a fourth one called inherited properties although this variety would apply only to objects in an observer.

6 If OOO does describe reality or, at the very least, is a useful analysis tool; what questions arise from its success?

6.1 Why are Objects useful Outside and Inside an Observer?

Objects outside and inside an observer are both useful for analysing interactions and that raises the question why objects with similar characteristics are useful in both cases? If objects work in an observer while we know that neural networks are the underlying architecture then we could expect a mapping between the networks and objects[15]. Trying to define this mapping looks useful because objects offer an accessible interface for humans while neural networks are often difficult to analyse.
The mirror effect of objects in several interacting observers and the mirroring of real external objects in objects in an observer lends some credence to the idea that the evolution of observer cognisance has been driven by real objects[16].
In the OOP C# language an "abstract" object, meaning an object which cannot be instantiated, is often used to define functions, properties and interfaces all child objects have (to have). A next step is to define a real abstract object with properties which are inherited by all real objects.

---

14 The Weber number may be written as:

$$We = \frac{\text{Inertial pressure}}{\text{Laplace pressure}} = \frac{\rho v^2}{(\sigma/l)} = \frac{\rho v^2 l}{\sigma}$$

where

- ρ is the density of the fluid (kg/m3).
- v is its velocity (m/s).
- l is its characteristic length, typically the droplet diameter (m).
- σ is the surface tension (N/m).

15 The brain of humans are flexible and if an object layer is used it will just be one of several tactics used for processing information. For example an object approach is less probable when automated processes apply like hitting a golf ball or when practicing 'het kluwen-breien' which is a dutch term for chatting while knitting.

16 A 'mirror' effect will also have been noticed by Object Oriented Programmers and some application users, when an object model matches concrete objects and processes. I have noticed how some application users start intuiting the object structure of a program and write support documentation which mirrors the objects in the application and in reality.

6.2 An 'interface' Object which Observers use when Interpreting Reality

An 'interface' object which observers use when interpreting reality, was introduced in the dogs script; for the human and the dog. Another similar type of ‘worker’ object could be responsible for 'priming' existing memorized objects when an external object is recognised. Another ‘worker’ object could also be responsible for updating existing objects if a new property occurs during an experience, see above: Nida can "jump 2 meters into the air". It looks improbable that this could work in inanimate objects like a water drop. Identification of different types of object would suggest that a flat ontology is less likely to apply when including objects within observers.

7 Conclusion

A limited number of OOO characteristics have been addressed using WOOOF to write and display scenarios representing interactions between objects.
The three scenarios without an observer illustrate drop coalescence, a golf club hitting a ball and a stone hitting the surface of a pool and sinking to the bottom of the pool. They have a uniform approach: Two types of time objects, called Timeline and Event, play a role with no explicit parallel in OOO. The OOO idea of vicarious causation has been implemented in the instantaneous Event. The Event simplifies scenarios because it has an input with no more than two objects and an output, also, with no more than two objects. Application to scenarios with many Events such as Brownian motion or atomisation instead of coalescence of water drops may not be practical in WOOOF but could be a part of (a speculative) reality.
Finding a generic approach to interactions looks promising and may lead to a ‘pattern’ which can be applied to any scenario.
The attempts to standardize and simplify Compound object formation and disintegration, ran into a problem with avoidance of an arbitrary ratio of relative size. When should the object formed by a collision be a new object and when should it be a modified version of one of the original objects? Allowing SOs of all combinations of real objects in a compound object is way of avoiding this type of arbitrary boundary.
Finding a practical level of detail in all types of object (inanimate, Timeline, Event) is a decision which is relevant when developing a scenario. The OOO idea of withdrawal of real objects is one guide to finding a practical level of detail in scenarios.
The scenarios include a Real Object in its Sensual Object making use of the OOP concept of information hiding to implement withdrawal. Testing with a scenario where a Real Object requires more than one Sensual Object, because it is interacting with several other objects, looks like a logical next step.
A speculative application of OOO to the processing of objects within an observer leads to several interesting complications. WOOOF lends itself to showing parallel timelines to represent delays between observers. Comparing the observer scenario with the other scenarios emphasises that in the latter, Events involve physical interaction, such as collision, while observers have a wider range of communication.
Information in an observer uses inheritance and adds an object type containing properties which, for example, apply to all dogs or an object animal which applies to all dogs and humans.
The observer scenario includes examples of information processing using ‘worker’ objects which create an interface between external objects and objects within an observer. These are another example of objects which differ from objects rooted in reality and at odds with the OOO idea of a flat ontology.

8 Appendix: Brownian Motion

A particle of silica is suspended in water at 293K.

Radius of a particle of silica.	5,00E-07 m
Density of silica.	2,20E+03 Kg/m3
Mass of silica particle.	1,15E-15 Kg

The particle is bombarded by water particles at what is called the **Total Collision Rate**. A molecule of water collides with and bounces off the particle in a finite time called the **Collision Time**. The collision is further influenced by the interaction between water molecules and the density of molecules in water. When air molecules replace water there is no significant interaction between air molecules at the surface of the particle.

Total collision rate	1,00E+19 per second

Collision Time	1,00E-12 seconds

We can calculate the number of water molecules interacting with the particle at a given moment and the percentage of this mass relative to the mass of the silica particle.

Number of water molecules in contact with particle at any time.	1,00E+07
Mass of water in contact with particle.	2,99E-19 Kg
% of water mass / particle mass.	0,03%

The Fluctuation Dissapation Theorem recognises that the movement of a particle of silica or a water molecule must equal the energy dissapated by friction due to the viscosity as the particle or molecule moves through the molecules of water. The latter follows Stokes law.
The result is that the energy of any particles is 0,5 * k_b * T where k_b is Bolzmanns constant.

Bolzmanns Constant $k_{b.}$	1,3806E-23 J/K
Temperature.	293 K
Average energy of silica particle (0,5 * k *T per degree of freedom).	6,067737E-21 J
Energy of a single water molecule	6,067737E-21 J

The pollen and the water molecules have the same energy because they are at the same temperature.
And from the energy and mass we can work out the velocity of the particle and the molecules.

Thermal velocity of the silica particle	3,25E-03 m/sec
Thermal velocity of the water molecule	6,37E+02 m/sec

Velocity autocorrelation time is the time it takes for the pollen particle to lose all the energy from a single collision and may be a useful time unit.

Viscosity of water	1,00E-03 Pa s
The drag of water on the particle (6*pi*Viscosity * R)	9,44E-09
Velocity autocorrelation time (Mass of particle / drag)	1,22E-07 Seconds

The Total Collision Rate (N) is extremely high but almost all the collisions cancel each other out. The N**et Collision Imbalance** is the instantaneous difference in the number of water molecules striking opposing sides of the particle. This statistical asymmetry creates a fluctuating net force that drives Brownian motion. This is also called the accumulation effect. The 3 billion excess uncompensated impacts pile up in a single direction. It is this relatively small statistical asymmetry of 3 billion uncompensated net collisions which produces the random walk of brownian motion.
To put it another way the time-averaged imbalance is exactly zero but the instantaneous variance is non-zero and dictates the particle's random movement. The absolute difference in the number of collisions between sides scales as sqrt(Total collision rate).

net collision imbalance = sqrt (Total Collision Rate)	3,16E+09 n/sec
The molecules causing the net imbalance are in contact with the particle for % of the time.	0,32%
Relative Imbalance: The percentage difference relative to total collisions scales as 1/net collision imbalance.	3,16E-10 n/sec

If we take the Velocity autocorrelation time as a reasonable time scale we need to take account of 10^{12} collisions and about 400 net collision imbalances.

Collisions in the Velocity autocorrelation time	1,22E+12 n/sec

net collision imbalance in Velocity autocorrelation time 3,86E+02 n/sec

An alternative is to simulate a single collision imbalance event. On average about 3160 million molecules hit the particle between any 2 imbalance events and simulation would require twice this number of events and compound object mutations.

Collisions per imbalance event. 3,16E+09 n/sec

Bibliography

[1] Object Oriented Ontology: A New Theory of Everything, Graham Harman, 2018
[2] Waves and Stones: On the Ultimate Nature of Reality, Graham Harman, 2025
[3] On Vicarious Causation,Graham Harman, 2007
[4] The Universe of Things, Stevan Shaviro, 2014
[4 location 61%] The Universe of Things, Stevan Shaviro, 2014